Adaptive Feature Selection for Predicting Video Rebuffering

Project Summary

Video rebuffering is one of the most disruptive quality-of-experience (QoE) events in streaming. Assignment 1 showed that time-windowed network features can infer a session's *current* resolution. Predicting *upcoming* rebuffering is both harder and more useful: a player or network that flags an at-risk session early can lower bitrate or reroute traffic before playback stalls. Computing a large feature set on every window is expensive, and many features add little signal for a given prediction.

Research question. Can adaptive feature selection reduce the computational cost of video rebuffering prediction while matching the performance of a model that always uses the full feature set?

The system will start from a cheap feature subset and request additional, more expensive features only when the current prediction is not confident enough. This sits in the course's suggested open-problem area of *the systems costs of different features*. It matters for real-time video analytics, where predictions must be made quickly and at scale.

Data

We will use the Assignment 1 video QoE materials: the Netflix capture and labeled session files in the course data repository (`data/video-qoe/`, `video_dataset.pkl`, `netflix_session.pkl`, and `netflix.pcap`). Features include NetML / SAMP windowed statistics and the segment-download-rate feature from Assignment 1.

The task is a **future-horizon binary prediction**: at time t, using only information available so far, predict whether a rebuffering event occurs in $(t, t+\Delta]$ (e.g., the next 10–30 seconds). Labels will come from available session fields where present, or be derived from buffer and segment-download dynamics (download rate remaining below playback rate long enough to exhaust the buffer). We will use a temporal train/test split so that future information cannot leak into training.

Machine Learning

Baselines. Random Forest and XGBoost trained on (1) the full feature set and (2) a fixed cheap subset (packet/byte counts and throughput only).

Adaptive method. A cascade / sequential-acquisition model: a cheap-feature classifier runs first; if the predicted probability is near the decision boundary, the system computes the next most informative feature(s) and reclassifies. Features are acquired in order of expected predictive value relative to measured extraction cost. Acquisition stops when confidence is high enough or a cost budget is reached.

We will compare the full-feature model, the fixed cheap subset, and the adaptive approach.

Evaluation

Prediction quality. Accuracy, precision, recall, F1, ROC-AUC, and a confusion matrix. Because rebuffering is likely rare, we will report class balance and emphasize recall and F1 on the positive class. We will also measure *lead time*: how early before a stall the model correctly flags the session.

Systems cost. Features computed per prediction, fraction of windows that escalate beyond the cheap subset, feature-extraction time, and inference time. We will sweep confidence thresholds to plot the accuracy–cost curve and identify operating points that stay close to full-model performance at substantially lower feature cost.

Learning Objective

We expect to learn how feature representation and selection affect both model quality and systems cost, and to practice designing an adaptive pipeline that spends compute only on hard examples. More broadly, the project is about a deployment tradeoff in real-time ML: maximizing accuracy is not the only objective when latency and feature cost also matter.